10f. SIMD: Branchless Code

Martin Alfaro
PhD in Economics

INTRODUCTION

SIMD accelerates workloads by applying a single instruction to many elements at once. Its effectiveness heavily hinges on all elements following the same control flow. Yet, this pattern can be disrupted when conditionals are introduced (either explicitly in the source code or implicitly by the compiler). Once data elements evaluate a condition differently, their execution paths diverge and the hardware can no longer maintain a unified instruction stream. While compilers attempt to mitigate this issue by transforming code into SIMD-friendly forms, these adaptations tend to introduce additional work that reduces the overall benefit of SIMD.

This section explores strategies for efficiently applying SIMD in the presence of conditional behavior. We'll first examine scenarios in which the compiler introduces branches as a byproduct of its internal computation techniques. By employing alternative coding strategies, we'll show how these branches can be avoided altogether, thereby preserving a uniform instruction stream.

After this, we'll explore conditional statements that are intrinsic to the program's logic and therefore unavoidable. These are the explicit conditions that users introduce to ensure that certain operations execute only under specific circumstances. Here, we'll revisit the usual approaches to expressing conditions, focusing on their internal implementation. The goal is to compare the relative strengths and limitations of each approach, indicating which ones interact more favorably with SIMD execution.

Finally, we'll show that conditional statements can be equivalently recast as algebraic operations, effectively removing the branching logic that disrupts SIMD execution. In several scenarios, the compiler may implement these transformations automatically when it's likely beneficial.

TYPE INSTABILITY AND BOUNDS CHECKING AS CONDITIONS TO AVOID

Certain coding patterns lead to hidden branching. These are conditional execution paths that the user doesn't write explicitly, but the compiler adds internally to carry out the computation.

One particular source of hidden branching is type instability. When a function can return values of different types, Julia must generate separate execution paths to handle each possibility. These extra branches remain invisible in the source code, yet they still break the uniform instruction flow that SIMD depends on. The solution is simply to restore type stability, for which we can apply all the techniques explored in Chapter 8 of this book. It's worth remarking, though, that type stability should

always be a priority, regardless of any SIMD consideration: any attempt to achieve high performance is nearly impossible without ensuring type stability. Considering this, we'll proceed throughout the section assuming that type stability was guaranteed.

A second source of hidden branching arises in for-loops due to bounds checking. This operation represents a subtle form of conditional execution, where each iteration is executed only when indices remain within valid bounds. Bound checks interfere with the application of SIMD, as it prevents the compiler from treating the for-loop as a uniform sequence of operations.

A consequence of this behavior is that **adding `@inbounds` is a prerequisite for the application of SIMD**. If bound checks remain in place, inserting `@simd` alone will rarely lead the compiler to emit SIMD instructions. In fact, merely adding `@inbounds` could be enough to enable SIMD on its own, rendering `@simd` annotations redundant. In those cases, you'll notice that using `@inbounds @simd` rather than `@inbounds` won't yield any additional speedup. Both of these points are demonstrated below. [1]

NO MACROS

```julia
x = rand(1_000_000)

function foo(x)
    output = similar(x)

    for i in eachindex(x)
        output[i] = 2/x[i]
    end

    return output
end
```

```
julia> @btime foo($x)
  1.684 ms (3 allocations: 7.629 MiB)
```

@SIMD

```julia
x = rand(1_000_000)

function foo(x)
    output = similar(x)

    @simd for i in eachindex(x)
        output[i] = 2/x[i]
    end

    return output
end
```

```
julia> @btime foo($x)
  1.683 ms (3 allocations: 7.629 MiB)
```

@INBOUNDS

```julia
x = rand(1_000_000)

function foo(x)
    output = similar(x)

    @inbounds for i in eachindex(x)
        output[i] = 2/x[i]
    end

    return output
end
```

```
julia> @btime foo($x)
  904.666 μs (3 allocations: 7.629 MiB)
```

@INBOUNDS + @SIMD

```julia
x = rand(1_000_000)

function foo(x)
    output = similar(x)

    @inbounds @simd for i in eachindex(x)
        output[i] = 2/x[i]
    end

    return output
end
```

```
julia> @btime foo($x)
  915.628 μs (3 allocations: 7.629 MiB)
```

A practical lesson from these examples is that, when signaling to the compiler that SIMD is appropriate, for-loops should be annotated with `@inbounds @simd` rather than using `@simd` alone.

> **Broadcasting and For-Loops**
>
> Broadcasting in Julia automatically disables bounds checking and applies SIMD instructions. This often makes broadcast versions appear faster than a straightforward for-loop. However, **broadcasting is just compact notation for implementing for-loops**. As a result, a for-loop annotated with `@inbounds @simd` usually achieves a similar performance to its broadcast variant, as demonstrated below.
>
> **BROADCASTING**
>
> ```julia
> x      = rand(1_000_000)
> foo(x) = 2 ./ x
> ```
>
> ```
> julia> @btime foo($x)
>  963.645 μs (3 allocations: 7.629 MiB)
> ```

FOR-LOOP (NO MACROS)

```
x      = rand(1_000_000)

function foo(x)
    output = similar(x)

    for i in eachindex(x)
        output[i] = 2/x[i]
    end

    return output
end
```

```
julia> @btime foo($x)
  1.928 ms (3 allocations: 7.629 MiB)
```

FOR-LOOP (OPTIMIZED WITH MACROS)

```
x      = rand(1_000_000)

function foo(x)
    output = similar(x)

    @inbounds @simd for i in eachindex(x)
        output[i] = 2/x[i]
    end

    return output
end
```

```
julia> @btime foo($x)
  1.011 ms (3 allocations: 7.629 MiB)
```

APPROACHES TO CONDITIONAL STATEMENTS

Certain workflows require the user to introduce conditional statements. When this occurs, conditions are integral to a program's logical flow and can't be avoided. This is why it's important to consider the most effective way to introduce conditions.

Regarding SIMD, a relevant distinction for conditional statements is whether they're evaluated eagerly or lazily. To illustrate, let's consider computing `1 + 1` only when certain condition `C` is met. A lazy strategy checks `C` first, performing the addition `1 + 1` only if the condition is true. Expressed differently, the operation is deferred until it's actually needed. By contrast, an eager approach performs `1 + 1` immediately, regardless of whether `C` is satisfied. If the condition eventually evaluates to `false`, the result of `1 + 1` is simply discarded.

When conditional statements are applied only once, a lazy approach is more performant as it avoids unnecessary work. Instead, a trade-off arises when the conditional appears inside a for-loop, as they may involve many potential computations. In this case, as SIMD can evaluate multiple operations simultaneously, it may be advantageous to compute all branches upfront, selecting the relevant results only afterward. This is especially true when the branches involve inexpensive computations.

In Julia, whether a conditional expression is evaluated lazily or eagerly depends on how the code is written. Next, we'll examine this aspect in more detail.

IFELSE VS IF

The `ifelse` function in Julia follows an eager evaluation strategy. This means that both the condition and all possible outcomes are computed before deciding which result to return. In contrast, the `if` construct favors lazy computations, only evaluating the branch corresponding to the condition's truth value.

The following example demonstrates the computational difference between `if` and `ifelse`. The example is based on a reduction operation, where the elements to be summed are contingent on a condition. Note that `ifelse` requires specifying an operation for both the true and false cases, we return zero when the condition introduced isn't met.

IF

```julia
x = rand(1_000_000)

function foo(x)
    output = 0.0

    for i in eachindex(x)
        if x[i] > 0.5
            output += x[i]/2
        end
    end

    return output
end
```

```
julia> @btime foo($x)
  1.097 ms (0 allocations: 0 bytes)
```

IF + SIMD

```julia
x = rand(1_000_000)

function foo(x)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        if x[i] > 0.5
            output += x[i]/2
        end
    end

    return output
end
```

```
julia> @btime foo($x)
  1.109 ms (0 allocations: 0 bytes)
```

IFELSE

```julia
x = rand(1_000_000)

function foo(x)
    output = 0.0

    for i in eachindex(x)
        output += ifelse(x[i] > 0.5, x[i]/2, 0)
    end

    return output
end
```

```
julia> @btime foo($x)
  969.382 µs (0 allocations: 0 bytes)
```

IFELSE + SIMD

```julia
x = rand(1_000_000)

function foo(x)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        output += ifelse(x[i] > 0.5, x[i]/2, 0)
    end

    return output
end
```

```
julia> @btime foo($x)
  234.668 µs (0 allocations: 0 bytes)
```

As the example reveals, the fact that `ifelse` is eager doesn't automatically trigger the application of SIMD. This is precisely why `@inbounds @simd` had to be included.

It's also worth noting that applying SIMD instructions with `ifelse` doesn't necessarily increase performance. The example below demonstrates this point.

IFELSE

```
x      = rand(5_000_000)
output = similar(x)

function foo!(output,x)
    for i in eachindex(x)
        output[i] = ifelse(x[i] > 0.5, x[i]/2, 0)
    end
end
```

```
julia> foo!($output,$x)
  45.187 ms (0 allocations: 0 bytes)
```

IFELSE + SIMD

```
x      = rand(5_000_000)
output = similar(x)

function foo!(output,x)
    @inbounds @simd for i in eachindex(x)
        output[i] = ifelse(x[i] > 0.5, x[i]/2, 0)
    end
end
```

```
julia> foo!($output,$x)
  40.844 ms (0 allocations: 0 bytes)
```

TERNARY OPERATORS

Ternary operators are an alternative approach to expressing conditional logic. They follow the syntax `<condition> ? <action if true> : <action if false>`. Unlike the previous methods, this form doesn't enforce an eager or a lazy approach. Instead, it relies on heuristics to determine which approach should be implemented. The decision depends on the compiler's assessment about which strategy will likely be faster in the application considered.

To illustrate this behavior, we revisit the same reduction example used earlier, now applying `@inbounds @simd` directly within each construct. In the following case, the ternary operator chooses an eager strategy, thus evaluating both expressions before selecting the appropriate result.

IF

```
x = rand(1_000_000)

function foo(x)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        if x[i] > 0.5
            output += x[i]/2
        end
    end

    return output
end
```

```
julia> foo!($x)
  1.052 ms (0 allocations: 0 bytes)
```

IFELSE

```
x = rand(1_000_000)

function foo(x)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        output += ifelse(x[i]>0.5, x[i]/2, 0)
    end

    return output
end
```

```
julia> foo!($x)
  288.050 µs (0 allocations: 0 bytes)
```

TERNARY

```
x = rand(1_000_000)

function foo(x)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        output += x[i]>0.5 ? x[i]/2 : 0
    end

    return output
end
```

```
julia> foo!($x)
  250.049 µs (0 allocations: 0 bytes)
```

Instead, the ternary operator opts for a lazy approach in the following example.

IF

```julia
x = rand(1_000_000)

function foo(x)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        if x[i] > 0.99
            output += log(x[i])
        end
    end

    return output
end
```

```
julia> foo!($x)
  977.767 µs (0 allocations: 0 bytes)
```

IFELSE

```julia
x = rand(1_000_000)

function foo(x)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        output += ifelse(x[i] > 0.99, log(x[i]), 0)
    end

    return output
end
```

```
julia> foo!($x)
  9.453 ms (0 allocations: 0 bytes)
```

TERNARY

```
x = rand(1_000_000)

function foo(x)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        output += x[i]>0.99 ? log(x[i]) : 0
    end

    return output
end
```

```
julia> foo!($x)
  1.094 ms (0 allocations: 0 bytes)
```

TERNARY OPERATOR COULD CHOOSE A LESS PERFORMANT APPROACH

It's worth noting that the method chosen by the ternary operator isn't foolproof. In the following example, the ternary operator ends up implementing the slower approach.

IF

```
x      = rand(5_000_000)
output = similar(x)

function foo!(output,x)
    @inbounds @simd for i in eachindex(x)
        if x[i] > 0.5
            output[i] = log(x[i])
        end
    end
end
```

```
julia> foo!($output,$x)
  63.213 ms (0 allocations: 0 bytes)
```

IFELSE

```
x      = rand(5_000_000)
output = similar(x)

function foo!(output,x)
    @inbounds @simd for i in eachindex(x)
        output[i] = ifelse(x[i] > 0.5, log(x[i]), 0)
    end
end
```

```
julia> foo!($output,$x)
  44.092 ms (0 allocations: 0 bytes)
```

TERNARY

```
x      = rand(5_000_000)
output = similar(x)

function foo!(output,x)
    @inbounds @simd for i in eachindex(x)
        output[i] = x[i]>0.5 ? log(x[i]) : 0
    end
end
```

```
julia> foo!($output,$x)
  62.488 ms (0 allocations: 0 bytes)
```

WHEN IS EACH APPROACH BETTER?

As a general guideline, conditional statements with **computationally-demanding operations will more likely benefit from a lazy implementation**. In contrast, **an eager approach is usually more performant when branches comprise simple algebraic computations**. These heuristics tend to drive the decision adopted by the ternary operator.

To make this concrete, consider a case where only one branch performs a computation and that computation is straightforward. Under these conditions, an eager evaluation combined with SIMD typically outperforms a lazy alternative. In fact, this is precisely the strategy adopted when using a ternary operator.

IF

```
x              = rand(1_000_000)
condition(a)   = a > 0.5
computation(a) = a * 2

function foo(x)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        if condition(x[i])
            output += computation(x[i])
        end
    end

    return output
end
```

```
julia> foo!($x)
  968.814 µs (0 allocations: 0 bytes)
```

IFELSE

```
x               = rand(1_000_000)
condition(a)    = a > 0.5
computation(a) = a * 2

function foo(x)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        output += ifelse(condition(x[i]), computation(x[i]), 0)
    end

    return output
end
```

```
julia> foo!($x)
  217.739 μs (0 allocations: 0 bytes)
```

TERNARY

```
x               = rand(1_000_000)
condition(a)    = a > 0.5
computation(a) = a * 2

function foo(x)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        output += condition(x[i]) ? computation(x[i]) : 0
    end

    return output
end
```

```
julia> foo!($x)
  272.096 μs (0 allocations: 0 bytes)
```

Instead, below we consider a branch with computational-intensive calculations. In this case, a lazy approach is faster, which is the approach implemented by the ternary operator.

When the conditional branch involves a computationally intensive operation, the situation reverses. Here, a lazy strategy avoids performing expensive work unnecessarily, which leads to better performance overall. Below, we illustrate a scenario like, showing that the ternary operator also chooses a lazy evaluation.

IF

```julia
x               = rand(2_000_000)
condition(a)    = a > 0.5
computation(a) = exp(a)/3 - log(a)/2

function foo(x)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        if condition(x[i])
            output += computation(x[i])
        end
    end

    return output
end
```

```
julia> foo!($x)
  31.847 ms (0 allocations: 0 bytes)
```

IFELSE

```julia
x               = rand(2_000_000)
condition(a)    = a > 0.5
computation(a) = exp(a)/3 - log(a)/2

function foo(x)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        output += ifelse(condition(x[i]), computation(x[i]), 0)
    end

    return output
end
```

```
julia> foo!($x)
  31.127 ms (0 allocations: 0 bytes)
```

TERNARY

```
x               = rand(2_000_000)
condition(a)    = a > 0.5
computation(a) = exp(a)/3 - log(a)/2

function foo(x)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        output += condition(x[i]) ? computation(x[i]) : 0
    end

    return output
end
```

```
julia> foo!($x)
  32.026 ms (0 allocations: 0 bytes)
```

VECTORS WITH CONDITIONS

In many situations, you may already have a vector of Boolean conditions available. This can happen because the condition vector is part of the dataset itself, or because the same conditions will be reused throughout the code, making it worthwhile to compute and store them in advance.

Julia represents such condition vectors using either `Vector{Bool}` or `BitVector`. The latter is Julia's default for broadcasted comparisons, as in expressions like `x .> 0`. While `BitVector` is memory-efficient and can be faster in some contexts, it can also inhibit SIMD optimizations. When this occurs, converting the `BitVector` to a `Vector{Bool}` can improve performance. The example below illustrates this effect: even after accounting for the cost of converting the type, the version using `Vector{Bool}` achieves a shorter execution time.

BITVECTOR

```
x         = rand(1_000_000)
bitvector = x .> 0.5

function foo(x,bitvector)
    output     = similar(x)


    @inbounds @simd for i in eachindex(x)
        output[i] = ifelse(bitvector[i], x[i]/i, x[i]*i)
    end

    return output
end
```

```
julia> foo($x,$bitvector)
  10.300 ms (3 allocations: 7.629 MiB)
```

VECTOR{BOOL}

```
x         = rand(1_000_000)
bitvector = x .> 0.5

function foo(x,bitvector)
    output     = similar(x)
    boolvector = Vector(bitvector)

    @inbounds @simd for i in eachindex(x)
        output[i] = ifelse(boolvector[i], x[i]/i, x[i]*i)
    end

    return output
end
```

```
julia> foo($x,$bitvector)
  2.417 ms (5 allocations: 8.583 MiB)
```

No Vector Holding Conditions

When the vector of conditions isn't already available, the conclusions above no longer apply. If your goal is to leverage SIMD, the most efficient strategy is to avoid building a condition vector altogether, and instead rely on `ifelse` applied directly to scalar values inside the for-loop. This approach eliminates unnecessary memory allocations, while still allowing the compiler to generate SIMD instructions effectively. The following example demonstrates this point.

BITVECTOR

```
x = rand(1_000_000)

function foo(x)
    output     = similar(x)
    bitvector  = x .> 0.5


    @inbounds @simd for i in eachindex(x)
        output[i] = ifelse(bitvector[i], x[i]/i, x[i]*i)
    end

    return output
end
```

```
julia> foo($x)
  11.126 ms (7 allocations: 7.749 MiB)
```

VECTOR{BOOL}

```
x = rand(1_000_000)

function foo(x)
    output     = similar(x)
    boolvector = Vector{Bool}(undef,length(x))
        boolvector .= x .> 0.5

    @inbounds @simd for i in eachindex(x)
        output[i] = ifelse(boolvector[i], x[i]/i, x[i]*i)
    end

    return output
end
```

```
julia> foo($x)
  1.842 ms (5 allocations: 8.583 MiB)
```

NO VECTOR

```
x = rand(1_000_000)

function foo(x)
    output     = similar(x)



    @inbounds @simd for i in eachindex(x)
        output[i] = ifelse(x[i]>0.5, x[i]/i, x[i]*i)
    end

    return output
end
```

```
julia> foo($x)
  1.210 ms (3 allocations: 7.629 MiB)
```

ALGEBRAIC OPERATIONS AS COMPOUND CONDITIONS

We can often eliminate branches by rewriting logical conditions using simple algebraic identities. By doing so, we're eliminating the hurdle due to branching that impedes a direct application of SIMD.

To see how this works, let's first express the logic in mathematical terms. Given a set $\{b_i\}_{i=1}^n$ where $b_i \in \{0,1\}$, two frequent constructs can be expressed:

- all conditions are satisfied when

$$\prod_{i=1}^{n} c_i = 1$$

- least one condition is satisfied when

$$1 - \prod_{i=1}^{n}(1 - c_i) = 1$$

These identities translate directly into Boolean expressions in Julia. Given two Boolean scalars `c1` and `c2`:

- `c1 && c2` becomes `Bool(c1 * c2)`,
- `c1 || c2` becomes `Bool(1 - !c1 * !c2)`.

Once you adopt this perspective, you can rewrite control flow in a way that's friendlier to vectorization. For example, you can replace branching logic in a for-loop as follows.

&&

```
x = rand(1_000_000)
y = rand(1_000_000)


function foo(x,y)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        if (x[i] > 0.3) && (y[i] < 0.6) && (x[i] > y[i])
            output += x[i]
        end
    end

    return output
end
```

```
julia> foo($x,$y)
  5.536 ms (0 allocations: 0 bytes)
```

ALGEBRAIC &&

```julia
x = rand(1_000_000)
y = rand(1_000_000)


function foo(x,y)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        if (x[i] > 0.3)  * (y[i] < 0.6)  * (x[i] > y[i])
            output += x[i]
        end
    end

    return output
end
```

```
julia> foo($x,$y)
  2.174 ms (0 allocations: 0 bytes)
```

||

```julia
x = rand(1_000_000)
y = rand(1_000_000)


function foo(x,y)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        if (x[i] > 0.3) || (y[i] < 0.6) || (x[i] > y[i])
            output += x[i]
        end
    end

    return output
end
```

```
julia> foo($x,$y)
  7.377 ms (0 allocations: 0 bytes)
```

ALGEBRAIC ||

```
x = rand(1_000_000)
y = rand(1_000_000)


function foo(x,y)
    output = 0.0

    @inbounds @simd for i in eachindex(x)
        if Bool(1 - !(x[i] > 0.3)  * !(y[i] < 0.6)  * !(x[i] > y[i]))
            output += x[i]
        end
    end

    return output
end
```

```
julia> foo($x,$y)
  2.177 ms (0 allocations: 0 bytes)
```

With broadcasting, we can reexpress the conditional logic as follows.

&&

```
x        = rand(1_000_000)
y        = rand(1_000_000)

foo(x,y) = @. ifelse((x>0.3) && (y<0.6) && (x>y), x,y)
```

```
julia> foo($x,$y)
  12.407 ms (3 allocations: 7.629 MiB)
```

ALGEBRAIC &&

```
x        = rand(1_000_000)
y        = rand(1_000_000)

foo(x,y) = @. ifelse((x>0.3) *  (y<0.6) *  (x>y), x,y)
```

```
julia> foo($x,$y)
  1.168 ms (3 allocations: 7.629 MiB)
```

||

```
x        = rand(1_000_000)
y        = rand(1_000_000)

foo(x,y) = @. ifelse((x>0.3) || (y<0.6) || (x>y), x,y)
```

```
julia> foo($x,$y)
  8.729 ms (3 allocations: 7.629 MiB)
```

ALGEBRAIC ||

```
x        = rand(1_000_000)
y        = rand(1_000_000)

foo(x,y) = @. ifelse(Bool(1 - !(x>0.3) * !(y<0.6) * !(x>y)), x,y)
```

```
julia> foo($x,$y)
  1.241 ms (3 allocations: 7.629 MiB)
```

FOOTNOTES

[1.] Recall that the compiler may automatically disable bounds checking in simple scenarios. For instance, this is the case when only `x` is indexed and `eachindex(x)` is employed as the iteration range. Instead, explicit use of `@inbounds` becomes necessary when both `x` and another vector indexed, as in the examples we introduce below.